

Technische Hochschule Ingolstadt
Fakultät Elektro- und Informationstechnik

Sensor Network Technologies and Sensor Fusion

Extended Kalman Filter
Unscented Kalman Filter

Nov 20, 2025

Welcome to the lecture!

Prof. Dr.-Ing. Björn Giesler

Technische Hochschule Ingolstadt

AIMotion, AI and Sensor Data Fusion

Email: bjoern.giesler@thi.de

- **Please register for the lecture on Moodle!**
  - Link: https://moodle.thi.de/course/view.php?id=10580
  - Key: **SensNT+DF#WS25**
- **News on Moodle:**
  - Slides for the lecture (upcoming lectures marked as WIP and are subject to change!)
  - Formulary, Exercises (Solutions provided after discussion in the lecture)
  - Exercises are optional, one task / sheet per week, but are highly recommended.
    They can be handed in for correction.
  - I will *try* to publish exercises before Wednesday‘s lecture so that initial questions can be discussed on Thursday.
- **Exam: Written exam, 90 minutes**
- **Consultation hours: Please arrange via e-mail to bjoern.giesler@thi.de.**
- **Please also don‘t hesitate to send questions, lecture feedback, change requests, etc.!**

- **In the last lectures we**
  - looked at the "most basic" variant of the Kalman Filter,
  - interrelated it to Bayesian estimators,
  - and looked at reasons why linear filters are not enough.
- **Today, we will**
  - study the Extended and Unscented Kalman Filter variants, and
  - start to check out how to use Kalman Filters for Multi-Object Tracking (MOT).

Extended Kalman Filter

- **We have employed the Markov assumption in the derivation of the EKF as Bayesian estimator.**
- **We will do the same for the nonlinear case:**

$$\hat{\mathbf{x}}_{t+1} = f(\hat{\mathbf{x}}_t, \mathbf{w}_t)$$
$$\mathbf{z}_t = g(\mathbf{x}_t, \mathbf{v}_t).$$

  - f is a (possibly nonlinear) dynamics function.
  - g is a (possibly nonlinear) measurement or output function.
  - $\mathbf{w}_0, \mathbf{w}_1, \ldots, \mathbf{v}_0, \mathbf{v}_1, \ldots$ are independent.
- **The nonlinear filtering problem is the determination of**

$$\hat{\mathbf{x}}_{t|t-1} = E(\mathbf{x}_t | \mathbf{y}_0, \ldots, \mathbf{y}_{t-1}),$$
$$\hat{\mathbf{x}}_{t|t} = E(\mathbf{x}_t | \mathbf{y}_0, \ldots, \mathbf{y}_t)$$

- **The general nonlinear filtering solution involves a Partial Differential Equation estimator... this is too much work and makes our brain hurt.**

- **The Extended Kalman Filter (EKF) is a heuristic for nonlinear filtering problems.**
- **It is based on**
  - linearizing the dynamics and measurement functions around the current estimate
  - propagating an approximation of the conditional expectation and covariance.
- **It often works well (when tuned properly)**
  - the more "locally linear" a process is (e.g. rotations tend to be good candidates)
  - the higher the sampling frequency
  - the better the estimate (chicken—egg problem)
- **Sometimes it doesn't –**
  - if your estimate isn't good, linearization around the estimate isn't great
  - if your process is highly nonlinear.

- **Consider a measurement function** $h : \mathbb{R}^n \to \mathbb{R}^m$.
- **Suppose** $E[\mathbf{x}] = \bar{\mathbf{x}},\ E[(\mathbf{x} - \bar{\mathbf{x}})(\mathbf{x} - \bar{\mathbf{x}})^\top] = \Sigma_\mathbf{x}$ **and** $\mathbf{z} = h(\mathbf{x})$.
- **If** $\Sigma_\mathbf{x}$ **is small,** ϕ **is not *too* nonlinear, then we can approximate**

$$\mathbf{z} \approx \tilde{\mathbf{z}} = h(\bar{\mathbf{x}}) + D_h(\bar{\mathbf{x}})(\mathbf{x} - \bar{\mathbf{x}})$$

  where $D_h(\mathbf{x})$ **is the matrix of partial derivatives of** h **in** $\mathbf{x}$ **, i.e. the *Jacobian*.**

- **This gives an approximation for the mean and covariance of the nonlinear function of a random variable:**

$$\bar{\mathbf{z}} \approx h(\bar{\mathbf{x}})$$
$$\Sigma_\mathbf{z} \approx D_h(\bar{\mathbf{x}})\Sigma_\mathbf{x} D_h(\bar{\mathbf{x}}).$$

- **If** $\Sigma_\mathbf{x}$ **is not small compared to the "curvature" of** h **, these estimates are poor.**

Jacobian Matrix

- **The Jacobian of a vector-valued function of several variables is the matrix of all its first-order partial derivatives.**
- **Definition:**
  - Let $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ be a function such that each of its first-order partial derivatives exists on $\mathbb{R}^n$.
  - The function takes $\mathbf{x} = (x_1, \ldots, x_n)$ as input and produces $f(\mathbf{x}) = (f_1(\mathbf{x}), \ldots, f_m(\mathbf{x}))$ as output.
  - The Jacobian of f is the $m \times n$ matrix whose (i, j) entry is $\frac{\partial f_i}{\partial x_j}$:

$$J_f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \cdots \frac{\partial f}{\partial x_n} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

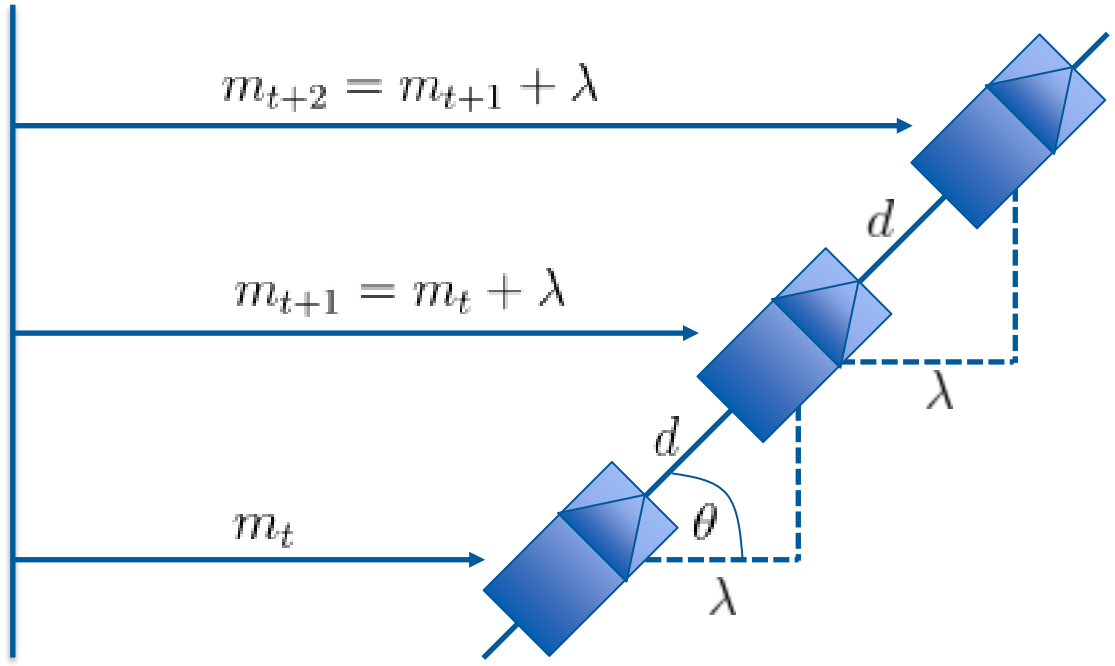
Extended Kalman Filter - Equations

- **For the prediction, we need the Jacobian of the state transition function:** $F_t = \frac{\partial f}{\partial x}(\hat{\mathbf{x}}_{t-1}, \mathbf{w}_{t-1})$
- **With this, we can formulate the prediction equations:**
  - Predicted state estimate: $\hat{\mathbf{x}}_{t+1} = f(\hat{\mathbf{x}}_t, \mathbf{w}_t).$
  - Predicted covariance estimate: $P_{t+1} = P'_t = F_t P_t F_t^\top + Q_t.$

- **For the update, we need the Jacobian of the measurement function:** $H_t = \frac{\partial h}{\partial x}(\hat{\mathbf{x}}_t) = \frac{\partial h}{\partial x}(f(\hat{\mathbf{x}}_{t-1}, \mathbf{w}_{t-1})).$
- **With this, we can formulate the update equations:**
  - Kalman Gain: $K_t = P'_t H_t^\top (H_t P'_t H_t \top + R)^{-1}$
  - State Update: $\hat{\mathbf{x}}_t = \mathbf{x'}_t + K_t(\mathbf{z}_t - h(\hat{\mathbf{x}}'_t))$
  - State Covariance Update: $P_t = (I - K_t H) P'_t$
- **Exactly the same as for the regular Kalman Filter, except for the dependency of H and F on t (by evaluating the Jacobian at current time).**

Simple Example for the Extended Kalman Filter (from [2]):

- **We are using a distance sensor to track an object moving in a straight line.**
- **The object moves at an angle to the distance sensor:**



$$m_{t+1} = m_t + \lambda$$
$$\lambda = d \cos\theta$$
$$d = v \cdot \Delta t$$

- **We model our state as** $\mathbf{x}_t = \begin{bmatrix} m_t \\ \theta \\ v \end{bmatrix}$

  and our state transition function as $f(\mathbf{x}_t) = (m_t + v \cdot \cos\theta \cdot \Delta t, \theta, v)^\top.$

- **The Jacobian of** f **is**

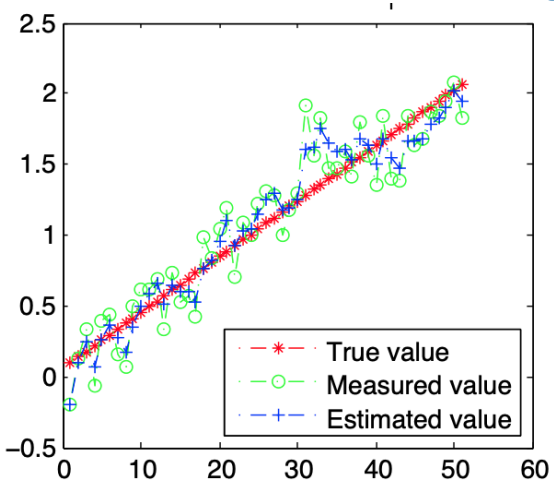
$$F = J_f = \begin{bmatrix} \partial(m_t + v\cos\theta\Delta t)/\partial m_t & \partial(m_t + v\cos\theta\Delta t)/\partial\theta & \partial(m_t + v\cos\theta\Delta t)/\partial v \\ \partial\theta/\partial m_t & \partial\theta/\partial\theta & \partial\theta/\partial v \\ \partial v/\partial m_t & \partial v/\partial\theta & \partial v/\partial v \end{bmatrix}$$

$$= \begin{bmatrix} 1 & -v\sin\theta\Delta t & \cos\theta\Delta t \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

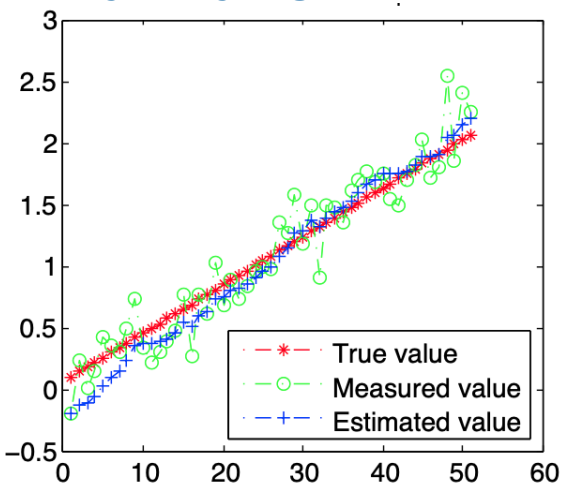
- **Since we measure the distance directly,** $z = h(\mathbf{x}_t) = m_t$ **and the Jacobian is**

$$H = J_h = \begin{bmatrix} \partial m_t/\partial m_t & \partial m_t/\partial\theta & \partial m_t/\partial v \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}.$$

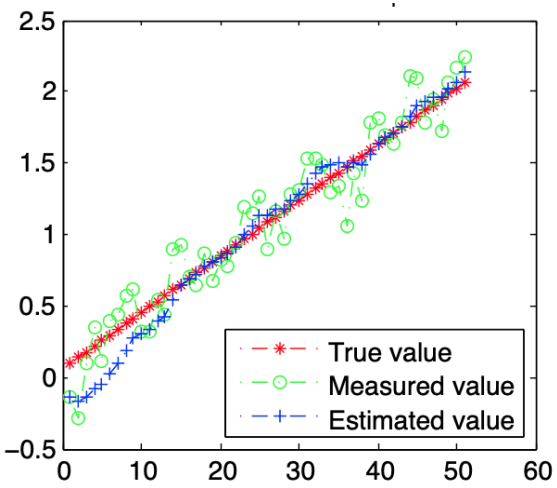
- **Experimental results by artificially varying measurement noise assumption R :**



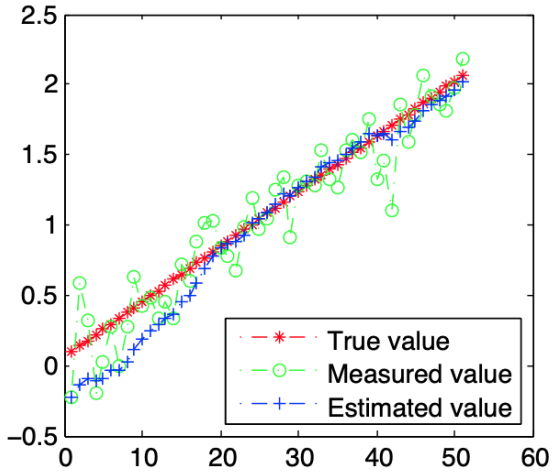
R=0.001 -- "trust the measurement"



R=0.05



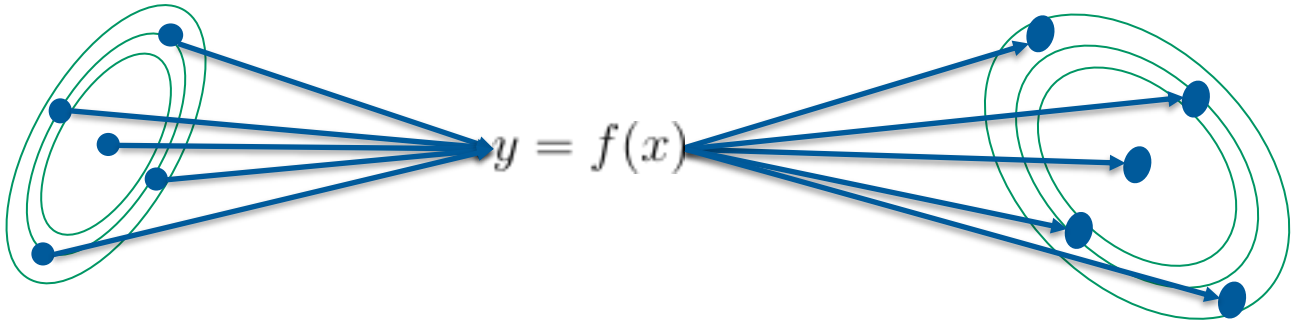
R=0.08



R=0.001 -- "trust the model"

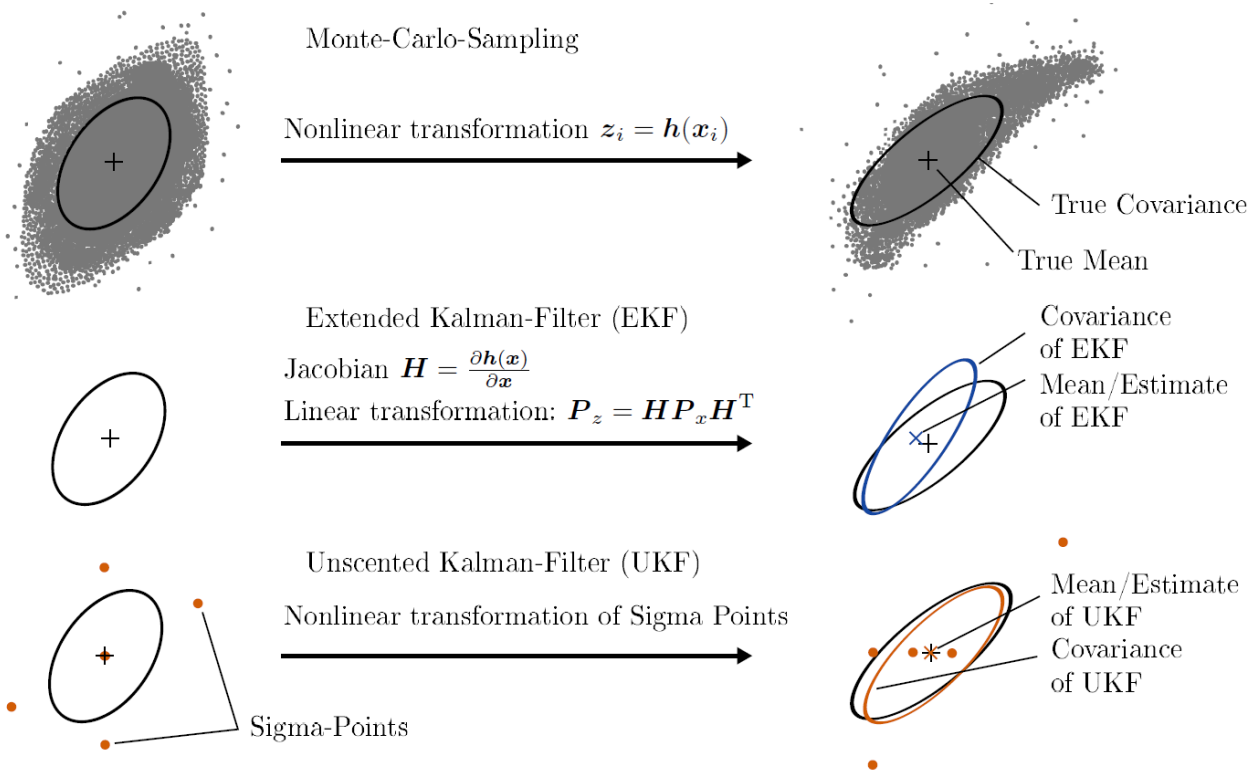
Unscented Kalman Filter

- **As we have seen, the Extended Kalman Filter helps us to get rid of *some* nonlinearities, provided they are reasonably well-behaved.**
- **It deteriorates for more complex functions.**
- **Enter the Unscented Kalman Filter. Basic idea:**
  - Sample the error distribution at *precisely chosen* points surrounding the mean
  - Transform these points through the state transition function
  - Use the transformed points to approximate the output distribution.



Unscented Kalman-Filter (UKF)

- Monte-Carlo Sampling:
  - Sampling of the probability density functions employing randomly chosen particles
  - High accuracy but high computational demands
- **Compromise:**
  - Assuming a normal distribution, the covariance is approximated employing deterministically chosen sigma points.
- How to choose the sigma points?
  - One is the mean (obviously)
  - Choose another **2n** points (n=state dimension) symmetrically around the mean based on the a priori covariance estimate.
  - Weigh them differently.



- **"Choose points symmetrically around the mean" – how do we do that?**
- **We used Eigenvectors to describe covariance matrices before, but those only transfer symmetry through *linear* transforms, and we're in non-linear space here.**
- **We remember that covariance matrices are symmetric and positive semi-definite:**

$$\Sigma = \Sigma^\top, \ \mathbf{x}^\top \Sigma \mathbf{x} \geq 0$$

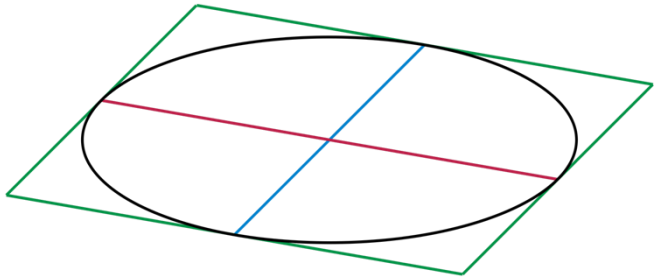
- **They are also positive definite if no diagonal element is zero:** $\mathbf{x}^\top \Sigma \mathbf{x} > 0$
- **Cholesky Decomposition: Any positive definite matrix can be written as**

$$\Sigma = LL^\top$$

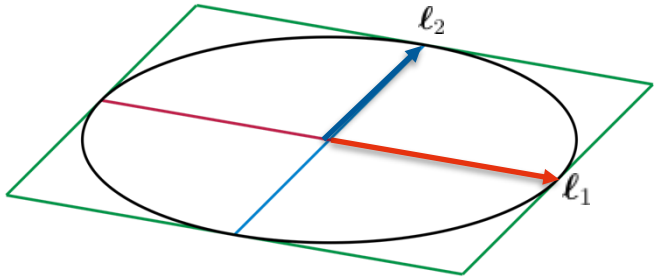
with L being a unique lower triangular matrix.

The Unscented Transform – Choosing Sigma Points

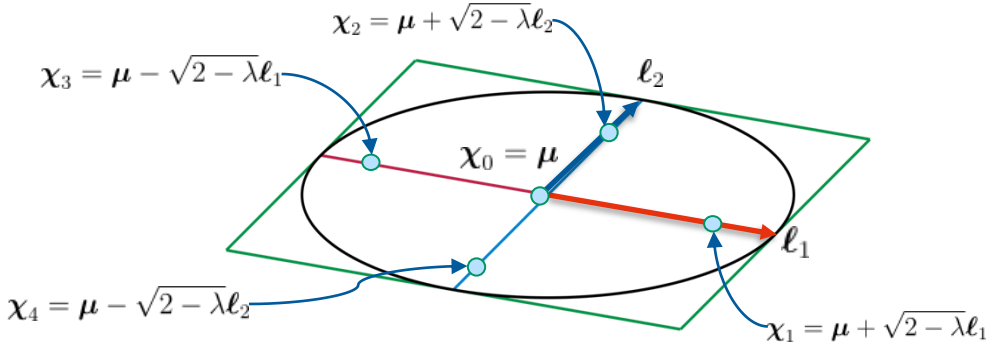
- **The Cholesky Decomposition is equivalent to a particular choice of conjugate axes of an ellipsoid.**
  - For an ellipsoid, two diameters are conjugate if and only if the tangent line to the ellipse at an endpoint of one diameter is parallel to the other diameter.
  - This is not the same as reconstructing the eigenvectors, but it gives symmetry axes that we can sample equidistant points on.



Two conjugate axes (red, blue) of an ellipsis Σ ...



...obtained as column vectors ℓ_i of the Cholesky Decomposition $\Sigma = LL^\top$...



...give the positions of the sigma points.

The Unscented Transform – Choosing Sigma Points

- We generate $2n + 1$ sigma points

$$\boldsymbol{\chi}_0 = \boldsymbol{\mu},$$

$$\boldsymbol{\chi}_i = \begin{cases} \boldsymbol{\mu} + \sqrt{n+\lambda} \cdot \boldsymbol{\ell}_i & \text{if } i = 1, \ldots, n \\ \boldsymbol{\mu} - \sqrt{n+\lambda} \cdot \boldsymbol{\ell}_{i-n} & \text{if } i = n+1, \ldots, 2n \end{cases}$$

where ℓ_i is the i-th column of the Cholesky decomposition matrix $L \in R^{n \times n}$

$$\boldsymbol{L} \cdot \boldsymbol{L}^{\mathrm{T}} = \boldsymbol{P}_{k|k}$$

and λ is a chooseable parameter.

- Every sigma point $\boldsymbol{\chi}_i$ gets assigned a corresponding weight w_i with

$$w_0 = \frac{\lambda}{n+\lambda}$$

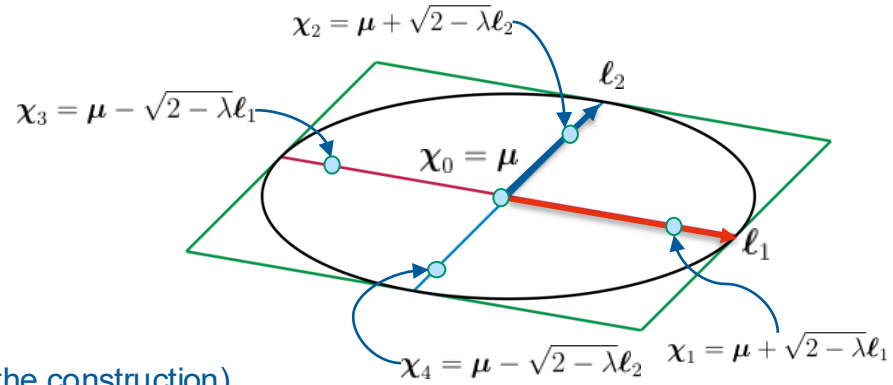
$$w_i = \frac{1}{2(n+\lambda)}, \quad i = 1 \ldots n$$

The Unscented Transform – Deriving Weights

- How do we derive the weights?
- We are mostly interested in the **mean** and the **variance** of the new covariance. The weights are a way to reconstruct those.

$$\text{Mean} = \mathbb{E}[\boldsymbol{\chi}] = \sum_{i=0}^{2n} P[\boldsymbol{\chi}_i]\boldsymbol{\chi}_i$$

$$\text{Variance} = \mathbb{E}[\boldsymbol{\chi}^2] = \sum_{i=0}^{2n} P[\boldsymbol{\chi}_i]\boldsymbol{\chi}_i^\top\boldsymbol{\chi}_i$$



- **Mean:** With the weights $w_0 = \frac{\lambda}{n+\lambda}$, $w_i = \frac{1}{2n+\lambda}$, we can compute

$$\bar{\boldsymbol{\chi}} = \sum_{i=0}^{2n} w_i\boldsymbol{\chi}_i \quad \text{(we know this from the construction)}$$

$$= [\frac{\lambda}{n+\lambda}\sum_{i=1}^{2n}\frac{1}{2(n+\lambda)}]\boldsymbol{\mu}$$

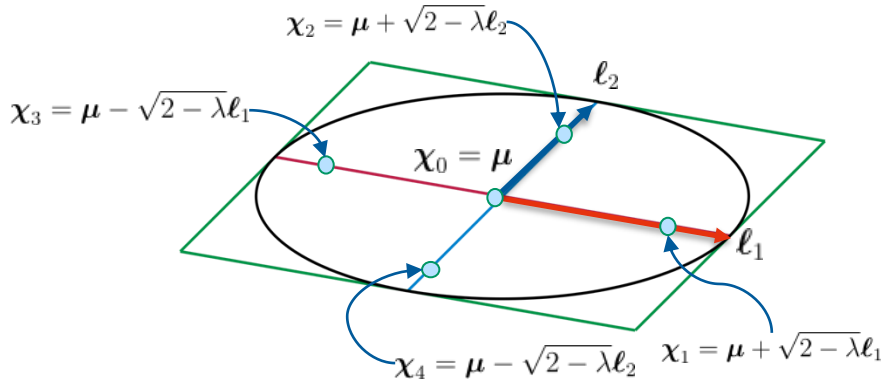
$$= [\frac{\lambda}{n+\lambda}2n\frac{1}{2(n+\lambda)}]\boldsymbol{\mu}$$

$$= \frac{n+\lambda}{n+\lambda}\boldsymbol{\mu} = \boldsymbol{\mu}.$$

The Unscented Transform – Deriving Weights

- **Variance:** With the weights $w_0 = \frac{\lambda}{n+\lambda}$, $w_i = \frac{1}{2n+\lambda}$, we can compute

$$
\begin{aligned}
S &= \sum_{i=0}^{2n} w_i(\boldsymbol{\chi}_i - \bar{\boldsymbol{\chi}})(\boldsymbol{\chi}_i - \bar{\boldsymbol{\chi}}) \\
&= \sum_{i=1}^{2n} w_i(\boldsymbol{\chi}_i - \bar{\boldsymbol{\chi}})(\boldsymbol{\chi}_i - \bar{\boldsymbol{\chi}}) \text{ because } \boldsymbol{\chi}_0 = \bar{\boldsymbol{\chi}} \\
&= \frac{1}{2(n+\lambda)} \sum_{i=1}^{2n} (\boldsymbol{\chi}_i - \bar{\boldsymbol{\chi}})(\boldsymbol{\chi}_i - \bar{\boldsymbol{\chi}}) \\
&= \frac{1}{2(n+\lambda)} \sum_{i=1}^{2n} (\boldsymbol{\mu} \pm \sqrt{n+\lambda}\boldsymbol{\ell}_{i\%n} - \boldsymbol{\mu})(\boldsymbol{\mu} \pm \sqrt{n+\lambda}\boldsymbol{\ell}_{i\%n} - \boldsymbol{\mu})^\top \\
&= \frac{1}{2(n+\lambda)} \sum_{i=1}^{2n} (\sqrt{n+\lambda}\boldsymbol{\ell}_{i\%n})(\sqrt{n+\lambda}\boldsymbol{\ell}_{i\%n})^\top \\
&= \frac{n+\lambda}{n+\lambda} \sum_{i=1}^{n} \boldsymbol{\ell}_i \boldsymbol{\ell}_i^\top = \Sigma.
\end{aligned}
$$



Unscented Kalman-Filter (UKF) – Prediction Step

1. Compute sigma-points $\boldsymbol{\chi}_{i,t|t}$ based on the current estimate $\hat{\boldsymbol{x}}_{t|t}$ and covariance $\boldsymbol{P}_{t|t}$.
2. Predict: Predict the sigma-points employing the nonlinear dynamics, i.e.

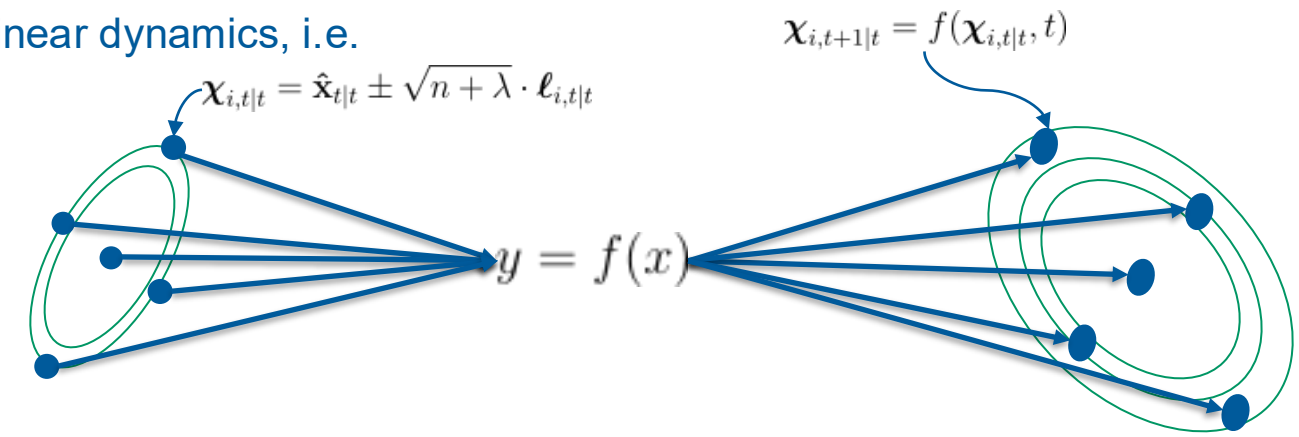
$$\boldsymbol{\chi}_{i,t+1|t} = f(\boldsymbol{\chi}_{i,t|t}, t)$$

3. Approximate the predicted mean

$$\hat{\mathbf{x}}_{t1+1|t} = \bar{\boldsymbol{\chi}} = \sum_{i=0}^{2n} w_i \boldsymbol{\chi}_{i,t+1|t}$$

and covariance:

$$P_{t+1|t} = Q_t + \sum_{i=0}^{2n} w_i (\boldsymbol{\chi}_{i,t+1|t} - \bar{\boldsymbol{\chi}})(\boldsymbol{\chi}_{i,t+1|t} - \bar{\boldsymbol{\chi}})^\top.$$



Unscented Kalman-Filter (UKF) – Measurement Update

1. Computing/Using sigma- points $\boldsymbol{\chi}_{i,t+1|t}$ based on the predicted estimate $\hat{\boldsymbol{x}}_{t+1|t}$ and covariance $\boldsymbol{P}_{t+1|t}$
2. Compute a predicted measurement from the weighted sum of predictions for each sigma point:

$$\hat{\mathbf{z}}_{t+1|t} = \sum_{i=0}^{2n} w_i \mathbf{z}_{i,t+1|t} \qquad \text{with} \qquad \mathbf{z}_{i,t+1|t} = h(\boldsymbol{\chi}_{i,t+1|t}, t)$$

3. Approximate the measurement covariance and the cross-covariance between state and measurement:

$$P_{zz} = \sum_{i=0}^{2n} w_i (\mathbf{z}_{i,t+1|t} - \hat{\mathbf{z}}_{t+1|t})(\mathbf{z}_{i,t+1|t} - \hat{\mathbf{z}}_{t+1|t})^\top$$

$$P_{xz} = \sum_{i=0}^{2n} w_i (\boldsymbol{\chi}_{i,t+1|t} - \bar{\boldsymbol{\chi}})(\mathbf{z}_{i,t+1|t} - \hat{\mathbf{z}}_{t+1|t})^\top$$

4. Compute the Kalman-Gain and perform correction of state and covariance.

$$K_t = P_{xz} \cdot (R + P_{zz})^{-1} \qquad \hat{\mathbf{x}}_{t+1|t+1} = \hat{\mathbf{x}}_{t+1|t} + K_t \cdot (\mathbf{z}_{t+1} - \hat{\mathbf{z}}_{t+1|t})$$

$$P_{t+1|t+1} = P_{t+1|t} - K_t P_{zz} K_t^\top .$$

Compare Kalman Filter and UKF Equations

	Linear	UKF
• Kalman Gain	$K_t = P_{t\|t-1} H^\top (H P_{t\|t-1} H^\top + R)^{-1}$	$K_t = P_{xz} \cdot (P_{zz} + R)^{-1}$
• Update Estimate	$\hat{\mathbf{x}}_{t+1\|t+1} = \mathbf{x}_{t+1\|t} + K_t \cdot (\mathbf{z}_{t+1} - H\hat{\mathbf{x}}_{t+1\|t})$	$\hat{\mathbf{x}}_{t+1\|t+} = \hat{\mathbf{x}}_{t+1\|t} + K_t \cdot (\mathbf{z}_{t+1} - \hat{\mathbf{z}}_{t+1\|t})$
• Update Covariance	$P_{t+1\|t+1} = P_{t+1\|t} - K_t H P_{t+1\|t}$	$P_{t+1\|t+1} = P_{t+1\|t} - K_t P_{zz} K_t^\top.$
• Predict	$\hat{\mathbf{x}}_{t+1\|t} = A\hat{\mathbf{x}}_{t\|t}$ $P_{t+1\|t} = A P_{t\|t} A^\top + Q$	$\hat{\mathbf{x}}_{t1+1\|t} = \bar{\chi}_{2n} = \sum_{i=0}^{2n} w_i \boldsymbol{\chi}_{i,t+1\|t}$ $P_{t+1\|t} = \sum_{i=0} w_i (\boldsymbol{\chi}_{i,t+1\|t} - \bar{\boldsymbol{\chi}})(\boldsymbol{\chi}_{i,t+1\|t} - \bar{\boldsymbol{\chi}})^\top + Q$

Interrelating Kalman, Extended Kalman, and Unscented Kalman Filters

	Input Error Distribution	Function Transform	Output Error Distribution
Kalman Filter	Approximate	Exact (linear)	Approximate
Extended Kalman	Approximate	Approximate	Approximate
Unscented Kalman	Approximate	Exact (nonlinear)	Approximate

UKF Advantage:

1. Nonlinear motion and sensor models do not need to be known.
2. No need to compute Jacobians.
3. Motion and sensor models need not be continuous or differentiable.
4. Computational complexity is similar to EKF.
5. Unscented Transform is accurate up to the 3rd moment for Gaussians.



Technische Hochschule Ingolstadt

Fakultät Elektro- und Informationstechnik

Thank you very much!